Day12 性能测试与系统优化

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 12 天（8 课时）
> 主题：性能测试与系统优化，找出系统瓶颈并优化

一、本日目标

1. 理解性能测试的意义和方法；
2. 掌握 Python 代码性能分析工具（cProfile、timeit）；
3. 掌握向量化优化技巧；
4. 掌握 pandas 性能优化技巧；
5. 掌握内存优化技巧；
6. 掌握并行计算（multiprocessing）；
7. 掌握缓存优化；
8. 完成系统性能优化实战。

二、知识要点

- 性能测试：测量代码执行时间、内存占用、CPU使用率；
- 性能分析工具：cProfile（函数级分析）、timeit（小段代码计时）、memory_profiler（内存分析）；
- 向量化优化：用 numpy/pandas 向量化操作替代 for 循环；
- pandas 优化：用 merge 替代循环匹配、用 groupby 替代逐行处理；
- 内存优化：选择合适的数据类型（int32替代int64、category替代object）；
- 并行计算：multiprocessing 多进程并行处理用户数据；
- 缓存优化：用字典缓存重复计算结果。

课时1：性能测试基础与工具

1.1 课时目标

理解性能测试的意义，掌握 Python 性能分析工具。

1.2 知识点深度讲解

1.2.1 为什么要做性能测试

性能测试的意义：
1. **发现瓶颈**：找出系统中最慢的部分；
2. **指导优化**：有针对性地优化瓶颈，而不是盲目优化；
3. **验证效果**：优化前后对比，确认优化有效；
4. **容量规划**：了解系统在不同数据量下的性能表现；
5. **用户体验**：确保系统在可接受时间内完成。

1.2.2 性能指标

1. **执行时间**：代码运行所需时间（秒）；
2. **CPU使用率**：CPU占用百分比；
3. **内存占用**：峰值内存使用量（MB）；
4. **吞吐量**：单位时间处理的数据量（行/秒）；
5. **响应时间**：从输入到输出的时间。

1.2.3 Python 性能分析工具

1. time 模块（最简单）

```
import time
start = time.time()
# 代码
elapsed = time.time() - start
print(f"耗时：{elapsed:.2f}秒")
```

2. timeit 模块（小段代码精确计时）

```
import timeit
# 测试代码执行1000次的平均时间
t = timeit.timeit("sum(range(1000))", number=1000)
print(f"平均耗时：{t/1000:.6f}秒")
```

3. cProfile（函数级性能分析）

```
import cProfile
cProfile.run("main()", sort="cumulative")
```

输出每个函数的调用次数、总时间、累计时间。

4. memory_profiler（内存分析，需安装）

```
from memory_profiler import profile
@profile
def my_func():
    # 代码
```

1.2.4 性能测试的原则

1. **先测量后优化**：不要凭感觉优化，先用工具测量；
2. **优化瓶颈**：80%的时间花在20%的代码上，优化那20%；
3. **保持正确性**：优化不能改变代码的功能和结果；
4. **对比测试**：优化前后都要测试，确认优化有效且没有引入新问题；
5. **考虑可读性**：不要为了微小性能提升牺牲代码可读性。

1.3 完整代码（性能测试工具演示）

```
# ============================================================
# 课时1：性能测试基础与工具
# ============================================================

import time
import timeit
import cProfile
import io
import pstats
import numpy as np
import pandas as pd



# ============================================================
# 1. time 模块：最简单的计时
# ============================================================
def demo_time():
    """演示 time 模块计时。"""
    print("=" * 60)
    print("1. time 模块计时")
    print("=" * 60)

    start = time.time()
    # 模拟耗时操作
    total = sum(range(1000000))
    elapsed = time.time() - start
    print(f"  sum(100万) 耗时：{elapsed:.4f}秒，结果：{total}")

    # 高精度计时
    start = time.perf_counter()
    total = sum(range(1000000))
    elapsed = time.perf_counter() - start
    print(f"  perf_counter 高精度：{elapsed:.6f}秒")



# ============================================================
```

```
# 2. timeit 模块：小段代码精确计时
# ============================================================
def demo_timeit():
    """演示 timeit 模块。"""
    print("\n" + "=" * 60)
    print("2. timeit 模块精确计时")
    print("=" * 60)

    # 测试 for 循环 vs 列表推导式
    setup = "data = list(range(10000))"

    # for 循环
    t_for = timeit.timeit(
        "result = []\nfor x in data:\n    result.append(x * 2)",
        setup=setup, number=1000
    )
    print(f"  for 循环 (1000次): {t_for:.4f}秒, 平均{t_for/1000:.6f}秒")

    # 列表推导式
    t_list = timeit.timeit(
        "[x * 2 for x in data]",
        setup=setup, number=1000
    )
    print(f"  列表推导式 (1000次): {t_list:.4f}秒, 平均{t_list/1000:.6f}秒")
    print(f"  列表推导式比 for 循环快: {t_for/t_list:.1f}倍")

    # numpy 向量化
    setup_np = "import numpy as np\ndata = np.arange(10000)"
    t_np = timeit.timeit(
        "data * 2",
        setup=setup_np, number=1000
    )
    print(f"  numpy向量化 (1000次): {t_np:.4f}秒, 平均{t_np/1000:.6f}秒")
    print(f"  numpy比for循环快: {t_for/t_np:.1f}倍")


# ============================================================
# 3. cProfile：函数级性能分析
```

```
# ============================================================
def slow_function():
    """模拟慢函数。"""
    total = 0
    for i in range(100000):
        total += i * 2
    return total


def fast_function():
    """模拟快函数。"""
    return np.sum(np.arange(100000) * 2)


def demo_cprofile():
    """演示 cProfile。"""
    print("\n" + "=" * 60)
    print("3. cProfile 函数级性能分析")
    print("=" * 60)

    # 运行 cProfile
    pr = cProfile.Profile()
    pr.enable()

    # 执行代码
    for _ in range(10):
        slow_function()
        fast_function()

    pr.disable()

    # 输出结果
    s = io.StringIO()
    ps = pstats.Stats(pr, stream=s).sort_stats("cumulative")
    ps.print_stats(10)  # 只显示前10个
    print(s.getvalue())
```

```
# ============================================================
# 4. 自定义计时器上下文管理器
# ============================================================
class Benchmark:
    """性能测试上下文管理器。"""

    def __init__(self, name="任务"):
        self.name = name
        self.start = None

    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, *args):
        elapsed = time.perf_counter() - self.start
        print(f"  [{self.name}] 耗时: {elapsed:.4f}秒")


def demo_benchmark():
    """演示自定义计时器。"""
    print("\n" + "=" * 60)
    print("4. 自定义计时器")
    print("=" * 60)

    with Benchmark("for循环求和"):
        total = 0
        for i in range(1000000):
            total += i

    with Benchmark("内置sum"):
        total = sum(range(1000000))

    with Benchmark("numpy求和"):
        total = np.sum(np.arange(1000000))


# ============================================================
```

```
# 主函数
# ============================================================
if __name__ == "__main__":
    print("=" * 70)
    print("  性能测试基础与工具演示")
    print("=" * 70)

    demo_time()
    demo_timeit()
    demo_cprofile()
    demo_benchmark()

    print("\n" + "=" * 70)
    print("  性能测试工具演示完成！")
    print("=" * 70)
```

1.4 运行结果与解读

运行后展示4种性能测试工具的使用和结果：
1. time 模块：简单计时；
2. timeit：精确计时，对比 for 循环/列表推导式/numpy 的性能差异；
3. cProfile：函数级分析，显示每个函数的调用次数和耗时；
4. 自定义计时器：方便的上下文管理器。

> 关键理解：
> 1. 性能测试先测量后优化，不要凭感觉；
> 2. timeit 适合小段代码精确计时；
> 3. cProfile 适合函数级性能分析，找出瓶颈；
> 4. numpy 向量化通常比 for 循环快几十倍。

1.5 当堂练习

1. 用 timeit 测试 `pd.Series.apply()` vs 向量化操作的性能差异；
2. 用 cProfile 分析 data_clean.py 的性能瓶颈；
3. 思考：为什么 numpy 向量化比 for 循环快？（提示：C语言底层实现+连续内存）

1.6 课时小结

- 性能测试意义：发现瓶颈、指导优化、验证效果；
- 性能指标：执行时间、CPU、内存、吞吐量；
- 工具：time（简单）、timeit（精确）、cProfile（函数级）、memory_profiler（内存）；
- 原则：先测量后优化、优化瓶颈、保持正确性、对比测试；
- numpy 向量化通常比 for 循环快几十倍。

课时2：系统性能瓶颈分析

2.1 课时目标

用 cProfile 分析整个系统的性能瓶颈。

2.2 知识点深度讲解

2.2.1 系统性能分析方法

分析整个系统的性能：
1. **全流程计时**：用 Timer 统计每个模块的耗时；
2. **函数级分析**：用 cProfile 分析每个函数的耗时；
3. **行级分析**：用 line_profiler 分析每行代码的耗时；
4. **内存分析**：用 memory_profiler 分析内存占用。

2.2.2 常见性能瓶颈

1. **for 循环**：逐行处理数据，速度慢；
2. **重复计算**：相同的计算重复执行；
3. **频繁IO**：频繁读写文件；
4. **内存不足**：数据量太大导致内存交换；
5. **低效算法**：时间复杂度高的算法。

2.3 完整代码（系统性能瓶颈分析）

```
# ============================================================
# 课时2：系统性能瓶颈分析
# ============================================================

import os
import sys
import time
import cProfile
import io
import pstats
import pandas as pd
import numpy as np

# 添加代码目录到路径
CODE_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\project_code"
sys.path.insert(0, CODE_DIR)

from config import CLEANED_DATA_FILE, STAGE1_TRAVEL_POINTS_FILE
from utils import Timer, log


def analyze_module_performance():
    """分析各模块的性能。"""
    print("=" * 70)
    print("  系统性能瓶颈分析")
    print("=" * 70)

    # ============================================================
    # 1. 数据读取性能
    # ============================================================
    print("\n1. 数据读取性能")
    print("-" * 60)

    with Timer("读取 cleaned_data.csv (dtype=str)"):
        df = pd.read_csv(CLEANED_DATA_FILE, dtype=str)
```

```
    print(f"  行数: {len(df)}, 列数: {len(df.columns)}")
    print(f"  内存占用: {df.memory_usage(deep=True).sum() / 1024 / 1024:.2f} MB")

    # ============================================================
    # 2. 常见操作性能对比
    # ============================================================
    print("\n2. 常见操作性能对比")
    print("-" * 60)

    # 准备数据
    df["lon"] = pd.to_numeric(df["lon"], errors="coerce")
    df["lat"] = pd.to_numeric(df["lat"], errors="coerce")
    df = df.dropna(subset=["lon", "lat"])

    # 2.1 逐行计算 vs 向量化计算
    print("\n2.1 距离计算：逐行 vs 向量化")
    sample = df.head(10000).copy()
    center_lon, center_lat = 123.4, 41.8

    # 逐行（apply）
    with Timer("逐行计算 (apply, 10000行)"):
        def calc_dist_row(row):
            from math import radians, sin, cos, sqrt, atan2
            R = 6371.0
            dlat = radians(row["lat"] - center_lat)
            dlon = radians(row["lon"] - center_lon)
            a = sin(dlat/2)**2 + cos(radians(center_lat)) * cos(radians(row["lat"])) *
sin(dlon/2)**2
            return R * 2 * atan2(sqrt(a), sqrt(1-a))
        sample["dist_row"] = sample.apply(calc_dist_row, axis=1)

    # 向量化（numpy）
    with Timer("向量化计算 (numpy, 10000行)"):
        lons = np.radians(sample["lon"].values)
        lats = np.radians(sample["lat"].values)
        clon = np.radians(center_lon)
        clat = np.radians(center_lat)
        dlat = lats - clat
```

```
        dlon = lons - clon
        a = np.sin(dlat/2)**2 + np.cos(clat) * np.cos(lats) * np.sin(dlon/2)**2
        sample["dist_vec"] = 6371.0 * 2 * np.arcsin(np.sqrt(a))

    # 验证结果一致
    max_diff = (sample["dist_row"] - sample["dist_vec"]).abs().max()
    print(f"  结果最大差异: {max_diff:.8f} km (验证一致)")

    # 2.2 groupby 性能
    print("\n2.2 分组统计：groupby")
    with Timer("groupby 统计每个用户的点数"):
        user_counts = df.groupby("imsi").size()
    print(f"  用户数: {len(user_counts)}, 平均点数: {user_counts.mean():.1f}")

    # 2.3 merge 性能
    print("\n2.3 数据合并：merge")
    station_df = pd.DataFrame({
        "laci": df["lac_id"].unique()[:100],
        "station_name": [f"基站{i}" for i in range(100)]
    })
    with Timer("merge 关联基站名"):
        merged = df.merge(station_df, left_on="lac_id", right_on="laci", how="left")
    print(f"  合并后行数: {len(merged)}, 匹配率:
{merged['station_name'].notna().mean()*100:.1f}%")

    # ============================================================
    # 3. cProfile 分析
    # ============================================================
    print("\n3. cProfile 函数级分析（距离计算）")
    print("-" * 60)

    def calc_all_distances():
        """计算所有点到中心的距离。"""
        lons = np.radians(df["lon"].values)
        lats = np.radians(df["lat"].values)
        dlat = lats - np.radians(center_lat)
        dlon = lons - np.radians(center_lon)
        a = np.sin(dlat/2)**2 + np.cos(np.radians(center_lat)) * np.cos(lats) *
```

```
np.sin(dlon/2)**2
        return 6371.0 * 2 * np.arcsin(np.sqrt(a))


    pr = cProfile.Profile()
    pr.enable()
    for _ in range(10):
        calc_all_distances()
    pr.disable()

    s = io.StringIO()
    ps = pstats.Stats(pr, stream=s).sort_stats("cumulative")
    ps.print_stats(10)
    print(s.getvalue())

    # ============================================================
    # 4. 内存分析
    # ============================================================
    print("\n4. 内存占用分析")
    print("-" * 60)
    print(f"  DataFrame 总内存: {df.memory_usage(deep=True).sum() / 1024 / 1024:.2f}
MB")
    print(f"  各列内存占用:")
    for col in df.columns:
        mem = df[col].memory_usage(deep=True) / 1024 / 1024
        dtype = df[col].dtype
        print(f"    {col:15s} ({str(dtype):10s}): {mem:.2f} MB")



if __name__ == "__main__":
    analyze_module_performance()
```

2.4 运行结果与解读

运行后展示：

1. 数据读取性能和内存占用；
2. 常见操作性能对比（逐行vs向量化、groupby、merge）；
3. cProfile 函数级分析；
4. 内存占用分析（各列内存）。

> 关键理解：
> 1. 逐行计算（apply）比向量化慢几十倍；
> 2. groupby 和 merge 是 pandas 高效操作，应该优先使用；
> 3. object 类型列（字符串）占用内存最多；
> 4. 系统瓶颈主要在逐行计算和重复IO。

2.5 当堂练习

1. 用 cProfile 分析 tr_stage2.py 的性能瓶颈；
2. 找出系统中最慢的3个操作；
3. 思考：如何优化这些瓶颈？（提示：向量化、缓存、并行）

2.6 课时小结

- 系统性能分析：全流程计时→函数级分析→行级分析→内存分析；
- 常见瓶颈：for循环、重复计算、频繁IO、内存不足、低效算法；
- 逐行计算（apply）比向量化慢几十倍；
- object类型列占用内存最多；
- 优化方向：向量化、缓存、并行、内存优化。

课时3：向量化优化

3.1 课时目标

掌握向量化优化技巧，用 numpy/pandas 向量化操作替代 for 循环。

3.2 知识点深度讲解

3.2.1 什么是向量化

向量化是指用数组操作替代逐元素循环：

```
# 非向量化（慢）
result = []
for x in data:
    result.append(x * 2)

# 向量化（快）
result = data * 2
```

向量化快的原因：
1. **C语言底层实现**：numpy 操作在C语言层面执行，避免Python解释器开销；
2. **连续内存**：numpy 数组在内存中连续存储，CPU缓存命中率高；
3. **SIMD指令**：现代CPU支持单指令多数据，一次处理多个元素；
4. **避免类型检查**：numpy 数组类型统一，不需要每次检查类型。

3.2.2 常见向量化操作

1. **算术运算**：`+ - * / **` 直接对数组操作；
2. **比较运算**：`> < == !=` 返回布尔数组；
3. **逻辑运算**：`& | ~` 对布尔数组操作；
4. **函数应用**：`np.sin() np.exp() np.log()` 等；
5. **聚合运算**：`sum() mean() max() min()` 等；
6. **广播机制**：不同形状数组的运算。

3.2.3 pandas 向量化

pandas 的 Series 和 DataFrame 也支持向量化：

```
# 列运算
df["new_col"] = df["col1"] * 2 + df["col2"]

# 条件赋值
df.loc[df["score"] > 60, "pass"] = True

# 字符串向量化
df["name_upper"] = df["name"].str.upper()

# 日期向量化
df["year"] = df["date"].dt.year
```

3.3 完整代码（向量化优化实战）

```
# ============================================================
# 课时3：向量化优化实战
# ============================================================

import time
import numpy as np
import pandas as pd



def demo_arithmetic_vectorization():
    """算术运算向量化。"""
    print("=" * 60)
    print("1. 算术运算向量化")
    print("=" * 60)

    n = 1000000
    data = np.random.rand(n)

    # 非向量化
    start = time.perf_counter()
    result1 = []
    for x in data:
        result1.append(x * 2 + 1)
    t1 = time.perf_counter() - start

    # 向量化
    start = time.perf_counter()
    result2 = data * 2 + 1
    t2 = time.perf_counter() - start

    print(f"  for循环: {t1:.4f}秒")
    print(f"  向量化: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  结果一致: {np.allclose(result1, result2)}")
```

```
def demo_distance_vectorization():
    """距离计算向量化（Haversine）。"""
    print("\n" + "=" * 60)
    print("2. 距离计算向量化（Haversine）")
    print("=" * 60)

    n = 100000
    lons = np.random.uniform(123.0, 124.0, n)
    lats = np.random.uniform(41.5, 42.5, n)
    center_lon, center_lat = 123.5, 42.0

    # 非向量化（逐行）
    start = time.perf_counter()
    from math import radians, sin, cos, sqrt, atan2
    R = 6371.0
    result1 = []
    for lon, lat in zip(lons, lats):
        dlat = radians(lat - center_lat)
        dlon = radians(lon - center_lon)
        a = sin(dlat/2)**2 + cos(radians(center_lat)) * cos(radians(lat)) *
sin(dlon/2)**2
        result1.append(R * 2 * atan2(sqrt(a), sqrt(1-a)))
    t1 = time.perf_counter() - start

    # 向量化
    start = time.perf_counter()
    lons_rad = np.radians(lons)
    lats_rad = np.radians(lats)
    clon_rad = np.radians(center_lon)
    clat_rad = np.radians(center_lat)
    dlat = lats_rad - clat_rad
    dlon = lons_rad - clon_rad
    a = np.sin(dlat/2)**2 + np.cos(clat_rad) * np.cos(lats_rad) * np.sin(dlon/2)**2
    result2 = R * 2 * np.arcsin(np.sqrt(a))
    t2 = time.perf_counter() - start

    print(f"  逐行计算: {t1:.4f}秒")
    print(f"  向量化: {t2:.4f}秒")
```

```
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  结果一致: {np.allclose(result1, result2)}")


def demo_pandas_vectorization():
    """pandas 向量化操作。"""
    print("\n" + "=" * 60)
    print("3. pandas 向量化操作")
    print("=" * 60)

    n = 100000
    df = pd.DataFrame({
        "score": np.random.randint(0, 100, n),
        "age": np.random.randint(18, 60, n),
        "name": [f"user_{i}" for i in range(n)]
    })

    # 3.1 列运算
    start = time.perf_counter()
    df["score_double"] = df["score"] * 2
    t1 = time.perf_counter() - start
    print(f"  列运算 (score*2): {t1:.4f}秒")

    # 3.2 条件赋值
    start = time.perf_counter()
    df["pass"] = False
    df.loc[df["score"] >= 60, "pass"] = True
    t2 = time.perf_counter() - start
    print(f"  条件赋值 (pass): {t2:.4f}秒, 通过率: {df['pass'].mean()*100:.1f}%")

    # 3.3 多条件
    start = time.perf_counter()
    df["level"] = "普通"
    df.loc[(df["score"] >= 80) & (df["age"] < 30), "level"] = "优秀青年"
    df.loc[(df["score"] >= 80) & (df["age"] >= 30), "level"] = "优秀中年"
    t3 = time.perf_counter() - start
    print(f"  多条件赋值 (level): {t3:.4f}秒")
    print(f"  level分布: {df['level'].value_counts().to_dict()}")
```

```
    # 3.4 字符串向量化
    start = time.perf_counter()
    df["name_upper"] = df["name"].str.upper()
    df["name_len"] = df["name"].str.len()
    t4 = time.perf_counter() - start
    print(f"  字符串向量化: {t4:.4f}秒")


def demo_broadcasting():
    """广播机制。"""
    print("\n" + "=" * 60)
    print("4. 广播机制")
    print("=" * 60)

    # 标量与数组
    a = np.array([1, 2, 3])
    print(f"  标量+数组: {a} + 1 = {a + 1}")

    # 不同形状数组
    a = np.array([[1, 2, 3], [4, 5, 6]])  # shape (2, 3)
    b = np.array([10, 20, 30])  # shape (3,)
    print(f"  (2,3) + (3,):\n{a + b}")

    # 列向量与行向量
    a = np.array([[1], [2], [3]])  # shape (3, 1)
    b = np.array([10, 20, 30])  # shape (3,)
    print(f"  (3,1) + (3,):\n{a + b}")

    # 实际应用：批量计算多个中心点的距离
    print(f"\n  实际应用：批量计算到多个中心点的距离")
    points = np.random.rand(100, 2)  # 100个点
    centers = np.random.rand(5, 2)  # 5个中心点
    # 广播：(100,1,2) - (5,2) -> (100,5,2)
    diffs = points[:, np.newaxis, :] - centers[np.newaxis, :, :]
    dists = np.sqrt((diffs ** 2).sum(axis=2))  # (100, 5)
    print(f"  距离矩阵形状: {dists.shape}")
    print(f"  每个点最近的中心: {np.argmin(dists, axis=1)[:10]}")
```

```
if __name__ == "__main__":
    print("=" * 70)
    print("  向量化优化实战")
    print("=" * 70)

    demo_arithmetic_vectorization()
    demo_distance_vectorization()
    demo_pandas_vectorization()
    demo_broadcasting()

    print("\n" + "=" * 70)
    print("  向量化优化实战完成！")
    print("=" * 70)
```

3.4 运行结果与解读

运行后展示4种向量化优化：
1. 算术运算向量化：加速几十倍；
2. 距离计算向量化（Haversine）：加速几十倍；
3. pandas 向量化操作：列运算、条件赋值、多条件、字符串向量化；
4. 广播机制：不同形状数组的运算，批量计算到多个中心点的距离。

> 关键理解：
> 1. 向量化是性能优化的第一选择，通常能加速几十倍；
> 2. numpy 向量化快的原因：C语言底层、连续内存、SIMD指令；
> 3. pandas 的 Series/DataFrame 也支持向量化，避免用 apply；
> 4. 广播机制可以批量计算多个点到多个中心的距离。

3.5 当堂练习

1. 把 purpose.py 中的逐行距离计算改为向量化；
2. 用广播机制批量计算所有点到所有商业区的距离；
3. 思考：什么情况下向量化不适用？（提示：逻辑复杂、依赖前一行结果）

3.6 课时小结

- 向量化：用数组操作替代逐元素循环，加速几十倍；

- 快的原因：C语言底层、连续内存、SIMD指令、避免类型检查；
- 常见操作：算术、比较、逻辑、函数、聚合、广播；
- pandas 向量化：列运算、条件赋值、字符串向量化、日期向量化；
- 广播机制：不同形状数组的运算，批量计算。

课时4：pandas 性能优化

4.1 课时目标

掌握 pandas 性能优化技巧。

4.2 知识点深度讲解

4.2.1 pandas 常见性能问题

1. **逐行迭代**：用 `iterrows()` 或 `apply()` 逐行处理；
2. **重复计算**：相同的分组/聚合重复执行；
3. **频繁拷贝**：不必要的 DataFrame 拷贝；
4. **低效合并**：用循环替代 merge；
5. **类型不当**：object 类型占用大量内存。

4.2.2 pandas 优化技巧

1. **用向量化替代 apply**：`df["col"].apply(func)` → 直接列运算；
2. **用 merge 替代循环匹配**：逐行查找 → merge；
3. **用 groupby 替代逐行分组**：手动分组 → groupby；
4. **用 query 替代布尔索引**：复杂条件用 query；
5. **用 category 类型**：低基数字符串列转 category；
6. **用 eval**：复杂表达式用 eval；
7. **链式操作**：避免中间变量，用方法链。

4.3 完整代码（pandas 性能优化实战）

```
# ============================================================
# 课时4：pandas 性能优化实战
# ============================================================

import time
import numpy as np
import pandas as pd


def demo_apply_vs_vectorized():
    """apply vs 向量化。"""
    print("=" * 60)
    print("1. apply vs 向量化")
    print("=" * 60)

    n = 100000
    df = pd.DataFrame({"a": np.random.rand(n), "b": np.random.rand(n)})

    # apply（慢）
    start = time.perf_counter()
    df["c_apply"] = df.apply(lambda row: row["a"] * 2 + row["b"] ** 2, axis=1)
    t1 = time.perf_counter() - start

    # 向量化（快）
    start = time.perf_counter()
    df["c_vec"] = df["a"] * 2 + df["b"] ** 2
    t2 = time.perf_counter() - start

    print(f"  apply: {t1:.4f}秒")
    print(f"  向量化: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  结果一致: {np.allclose(df['c_apply'], df['c_vec'])}")


def demo_iterrows_vs_itertuples():
    """iterrows vs itertuples。"""
```

```
    print("\n" + "=" * 60)
    print("2. iterrows vs itertuples")
    print("=" * 60)

    n = 10000
    df = pd.DataFrame({"a": np.random.rand(n), "b": np.random.rand(n)})

    # iterrows（最慢）
    start = time.perf_counter()
    result1 = []
    for idx, row in df.iterrows():
        result1.append(row["a"] + row["b"])
    t1 = time.perf_counter() - start

    # itertuples（较快）
    start = time.perf_counter()
    result2 = []
    for row in df.itertuples():
        result2.append(row.a + row.b)
    t2 = time.perf_counter() - start

    # 向量化（最快）
    start = time.perf_counter()
    result3 = (df["a"] + df["b"]).tolist()
    t3 = time.perf_counter() - start

    print(f"  iterrows: {t1:.4f}秒")
    print(f"  itertuples: {t2:.4f}秒")
    print(f"  向量化: {t3:.4f}秒")
    print(f"  itertuples比iterrows快: {t1/t2:.1f}倍")
    print(f"  向量化比iterrows快: {t1/t3:.1f}倍")


def demo_merge_vs_loop():
    """merge vs 循环匹配。"""
    print("\n" + "=" * 60)
    print("3. merge vs 循环匹配")
    print("=" * 60)
```

```
    n = 10000
    df1 = pd.DataFrame({"id": range(n), "value": np.random.rand(n)})
    df2 = pd.DataFrame({"id": range(n), "name": [f"name_{i}" for i in range(n)]})

    # 循环匹配（慢）
    start = time.perf_counter()
    name_map = dict(zip(df2["id"], df2["name"]))
    df1["name_loop"] = df1["id"].map(name_map)
    t1 = time.perf_counter() - start

    # merge（快）
    start = time.perf_counter()
    merged = df1.merge(df2, on="id", how="left")
    t2 = time.perf_counter() - start

    print(f"  map循环: {t1:.4f}秒")
    print(f"  merge: {t2:.4f}秒")
    print(f"  结果一致: {merged['name'].equals(df1['name_loop'])}")


def demo_category_optimization():
    """category 类型优化。"""
    print("\n" + "=" * 60)
    print("4. category 类型优化")
    print("=" * 60)

    n = 1000000
    categories = ["北京", "上海", "广州", "深圳", "杭州", "成都", "武汉", "西安"]
    df = pd.DataFrame({"city": np.random.choice(categories, n)})

    # object 类型
    mem_obj = df["city"].memory_usage(deep=True) / 1024 / 1024
    print(f"  object类型内存: {mem_obj:.2f} MB")

    # category 类型
    df["city_cat"] = df["city"].astype("category")
    mem_cat = df["city_cat"].memory_usage(deep=True) / 1024 / 1024
```

```
    print(f"  category类型内存: {mem_cat:.2f} MB")
    print(f"  内存节省: {(1 - mem_cat/mem_obj)*100:.1f}%")

    # 性能对比
    start = time.perf_counter()
    df.groupby("city")["city"].count()
    t1 = time.perf_counter() - start

    start = time.perf_counter()
    df.groupby("city_cat")["city_cat"].count()
    t2 = time.perf_counter() - start

    print(f"  groupby(object): {t1:.4f}秒")
    print(f"  groupby(category): {t2:.4f}秒")


def demo_query_optimization():
    """query 优化。"""
    print("\n" + "=" * 60)
    print("5. query 优化")
    print("=" * 60)

    n = 1000000
    df = pd.DataFrame({
        "a": np.random.randint(0, 100, n),
        "b": np.random.randint(0, 100, n),
        "c": np.random.rand(n)
    })

    # 布尔索引
    start = time.perf_counter()
    result1 = df[(df["a"] > 50) & (df["b"] < 30) & (df["c"] > 0.5)]
    t1 = time.perf_counter() - start

    # query
    start = time.perf_counter()
    result2 = df.query("a > 50 and b < 30 and c > 0.5")
    t2 = time.perf_counter() - start
```

```
    print(f"  布尔索引: {t1:.4f}秒, 结果{len(result1)}行")
    print(f"  query: {t2:.4f}秒, 结果{len(result2)}行")
    print(f"  结果一致: {len(result1) == len(result2)}")


if __name__ == "__main__":
    print("=" * 70)
    print("  pandas 性能优化实战")
    print("=" * 70)

    demo_apply_vs_vectorized()
    demo_iterrows_vs_itertuples()
    demo_merge_vs_loop()
    demo_category_optimization()
    demo_query_optimization()

    print("\n" + "=" * 70)
    print("  pandas 性能优化实战完成！")
    print("=" * 70)
```

4.4 运行结果与解读

运行后展示5种 pandas 优化：
1. apply vs 向量化：向量化快几十倍；
2. iterrows vs itertuples vs 向量化：向量化最快；
3. merge vs 循环匹配：merge更快更简洁；
4. category 类型：内存节省90%+，groupby更快；
5. query：复杂条件查询更简洁，性能相当。

4.5 课时小结

- pandas 优化：向量化替代apply、merge替代循环、groupby替代手动分组；
- category 类型：低基数字符串列转category，内存节省90%+；
- iterrows 最慢，itertuples 较快，向量化最快；
- query 适合复杂条件查询，代码更简洁。

课时5：内存优化

5.1 课时目标

掌握内存优化技巧，减少 DataFrame 内存占用。

5.2 知识点深度讲解

5.2.1 内存优化的意义

内存优化的意义：
1. **处理更大数据**：在有限内存下处理更多数据；
2. **提高速度**：内存占用少，CPU缓存命中率高，速度更快；
3. **避免OOM**：防止内存不足导致程序崩溃；
4. **降低成本**：不需要更大内存的服务器。

5.2.2 常见内存优化技巧

1. **选择合适数值类型**：int8/int16/int32 替代 int64，float32 替代 float64；
2. **category 类型**：低基数字符串列转 category；
3. **downcast**：用 pd.to_numeric 自动选择最小类型；
4. **稀疏类型**：大量空值的列用 SparseDtype；
5. **分块读取**：用 chunksize 分块处理大文件；
6. **删除不需要的列**：及时删除不用的列；
7. **inplace 操作**：用 inplace=True 避免拷贝。

5.3 完整代码（内存优化实战）

```
# ============================================================
# 课时5：内存优化实战
# ============================================================

import numpy as np
import pandas as pd



def demo_numeric_type_optimization():
    """数值类型优化。"""
    print("=" * 60)
    print("1. 数值类型优化")
    print("=" * 60)

    n = 1000000
    df = pd.DataFrame({
        "int64_col": np.random.randint(0, 100, n),  # 默认int64
        "float64_col": np.random.rand(n),  # 默认float64
    })

    # 原始内存
    mem_orig = df.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"  原始内存: {mem_orig:.2f} MB")
    print(f"    int64_col: {df['int64_col'].dtype},
{df['int64_col'].memory_usage(deep=True)/1024/1024:.2f} MB")
    print(f"    float64_col: {df['float64_col'].dtype},
{df['float64_col'].memory_usage(deep=True)/1024/1024:.2f} MB")

    # 优化后
    df_opt = df.copy()
    df_opt["int8_col"] = df["int64_col"].astype("int8")  # 0-100用int8
    df_opt["float32_col"] = df["float64_col"].astype("float32")  # 精度要求不高用
float32
    df_opt = df_opt.drop(columns=["int64_col", "float64_col"])

    mem_opt = df_opt.memory_usage(deep=True).sum() / 1024 / 1024
```

```
    print(f"\n  优化后内存: {mem_opt:.2f} MB")
    print(f"    int8_col: {df_opt['int8_col'].dtype},
{df_opt['int8_col'].memory_usage(deep=True)/1024/1024:.2f} MB")
    print(f"    float32_col: {df_opt['float32_col'].dtype},
{df_opt['float32_col'].memory_usage(deep=True)/1024/1024:.2f} MB")
    print(f"  内存节省: {(1 - mem_opt/mem_orig)*100:.1f}%")

    # 验证数据一致
    print(f"  int8与int64一致: {(df['int64_col'] == df_opt['int8_col']).all()}")
    print(f"  float32与float64最大差异: {(df['float64_col'] -
df_opt['float32_col']).abs().max():.8f}")


def demo_downcast():
    """downcast 自动优化。"""
    print("\n" + "=" * 60)
    print("2. downcast 自动优化")
    print("=" * 60)

    n = 1000000
    df = pd.DataFrame({
        "int_col": np.random.randint(0, 1000, n),
        "float_col": np.random.rand(n) * 100,
    })

    mem_orig = df.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"  原始内存: {mem_orig:.2f} MB")

    # downcast
    df_opt = df.copy()
    df_opt["int_col"] = pd.to_numeric(df_opt["int_col"], downcast="integer")
    df_opt["float_col"] = pd.to_numeric(df_opt["float_col"], downcast="float")

    mem_opt = df_opt.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"  优化后内存: {mem_opt:.2f} MB")
    print(f"  int_col类型: {df['int_col'].dtype} -> {df_opt['int_col'].dtype}")
    print(f"  float_col类型: {df['float_col'].dtype} -> {df_opt['float_col'].dtype}")
    print(f"  内存节省: {(1 - mem_opt/mem_orig)*100:.1f}%")
```

```
def demo_category_optimization():
    """category 优化。"""
    print("\n" + "=" * 60)
    print("3. category 优化")
    print("=" * 60)

    n = 1000000
    cities = ["北京", "上海", "广州", "深圳", "杭州", "成都", "武汉", "西安", "南京",
"重庆"]
    genders = ["男", "女"]
    df = pd.DataFrame({
        "city": np.random.choice(cities, n),
        "gender": np.random.choice(genders, n),
        "id": range(n),
    })

    mem_orig = df.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"  原始内存: {mem_orig:.2f} MB")
    print(f"    city(object): {df['city'].memory_usage(deep=True)/1024/1024:.2f} MB")
    print(f"    gender(object): {df['gender'].memory_usage(deep=True)/1024/1024:.2f}
MB")

    # 转 category
    df_opt = df.copy()
    df_opt["city"] = df_opt["city"].astype("category")
    df_opt["gender"] = df_opt["gender"].astype("category")

    mem_opt = df_opt.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"\n  优化后内存: {mem_opt:.2f} MB")
    print(f"    city(category): {df_opt['city'].memory_usage(deep=True)/1024/1024:.2f}
MB")
    print(f"    gender(category):
{df_opt['gender'].memory_usage(deep=True)/1024/1024:.2f} MB")
    print(f"  内存节省: {(1 - mem_opt/mem_orig)*100:.1f}%")
```

```
def demo_chunk_reading():
    """分块读取大文件。"""
    print("\n" + "=" * 60)
    print("4. 分块读取大文件")
    print("=" * 60)

    # 生成测试文件
    test_file = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result\large_test.csv"
    import os
    os.makedirs(os.path.dirname(test_file), exist_ok=True)

    # 生成100万行测试数据
    n = 1000000
    df = pd.DataFrame({
        "id": range(n),
        "value": np.random.rand(n),
        "category": np.random.choice(["A", "B", "C", "D"], n),
    })
    df.to_csv(test_file, index=False)
    file_size = os.path.getsize(test_file) / 1024 / 1024
    print(f"  测试文件: {file_size:.1f} MB, {n}行")

    # 一次性读取
    import time
    start = time.perf_counter()
    df_full = pd.read_csv(test_file)
    t_full = time.perf_counter() - start
    mem_full = df_full.memory_usage(deep=True).sum() / 1024 / 1024
    print(f"\n  一次性读取: {t_full:.2f}秒, 内存{mem_full:.1f} MB")

    # 分块读取（chunksize=100000）
    start = time.perf_counter()
    chunk_size = 100000
    total_rows = 0
    category_counts = {}
    for chunk in pd.read_csv(test_file, chunksize=chunk_size):
        total_rows += len(chunk)
```

```
        counts = chunk["category"].value_counts().to_dict()
        for k, v in counts.items():
            category_counts[k] = category_counts.get(k, 0) + v
    t_chunk = time.perf_counter() - start
    print(f"\n  分块读取(chunksize={chunk_size}): {t_chunk:.2f}秒")
    print(f"  总行数: {total_rows}")
    print(f"  category统计: {category_counts}")
    print(f"  分块读取内存占用远小于一次性读取（每次只有{chunk_size}行在内存）")


if __name__ == "__main__":
    print("=" * 70)
    print("  内存优化实战")
    print("=" * 70)

    demo_numeric_type_optimization()
    demo_downcast()
    demo_category_optimization()
    demo_chunk_reading()

    print("\n" + "=" * 70)
    print("  内存优化实战完成！")
    print("=" * 70)
```

5.4 运行结果与解读

运行后展示4种内存优化：
1. 数值类型优化：int8替代int64、float32替代float64，内存节省75%；
2. downcast：自动选择最小类型；
3. category：低基数字符串列转category，内存节省90%+；
4. 分块读取：大文件分块处理，内存占用小。

5.5 课时小结

- 内存优化：数值类型、category、downcast、分块读取；
- int8/int16/int32 替代 int64，float32 替代 float64；
- category 类型节省90%+内存；
- 分块读取处理大文件，避免OOM。

课时6：并行计算（multiprocessing）

6.1 课时目标

掌握 multiprocessing 多进程并行计算。

6.2 知识点深度讲解

6.2.1 为什么需要并行计算

Python 的 GIL（全局解释器锁）导致多线程无法真正并行执行CPU密集型任务。multiprocessing 多进程可以绕过 GIL，真正利用多核CPU。

适用场景：
1. **CPU密集型**：大量计算（如k-means聚类、距离计算）；
2. **独立任务**：每个用户的数据处理相互独立；
3. **数据可分**：可以按用户/时间分片处理。

6.2.2 multiprocessing 基本用法

```
from multiprocessing import Pool


def process_user(user_data):
    # 处理单个用户
    return result


# 创建进程池
with Pool(processes=4) as pool:
    results = pool.map(process_user, user_list)
```

6.2.3 注意事项

1. **Windows 下必须用** `if __name__ == "__main__":`；
2. **进程间通信开销**：数据传输有开销，任务太小不划算；
3. **共享数据**：用 Manager 或 Array 共享数据；
4. **进程数**：一般设为 CPU 核心数。

6.3 完整代码（并行计算实战）

```
# ============================================================
# 课时6：并行计算实战
# ============================================================

import time
import numpy as np
import pandas as pd
from multiprocessing import Pool, cpu_count


def cpu_intensive_task(n):
    """CPU密集型任务：计算大量平方根。"""
    total = 0
    for i in range(n):
        total += np.sqrt(i) ** 2
    return total


def demo_basic_multiprocessing():
    """基础 multiprocessing。"""
    print("=" * 60)
    print("1. 基础 multiprocessing")
    print("=" * 60)

    print(f"  CPU核心数: {cpu_count()}")

    tasks = [1000000] * 8  # 8个任务

    # 串行
    start = time.perf_counter()
    results_serial = [cpu_intensive_task(n) for n in tasks]
    t_serial = time.perf_counter() - start

    # 并行
    start = time.perf_counter()
    with Pool(processes=4) as pool:
```

```
        results_parallel = pool.map(cpu_intensive_task, tasks)
    t_parallel = time.perf_counter() - start

    print(f"  串行: {t_serial:.2f}秒")
    print(f"  并行(4进程): {t_parallel:.2f}秒")
    print(f"  加速比: {t_serial/t_parallel:.1f}倍")
    print(f"  结果一致: {results_serial == results_parallel}")


def process_user_kmeans(args):
    """处理单个用户的k-means聚类（用于并行）。"""
    imsi, user_df, k_min, k_max = args
    X = user_df[["lon", "lat"]].values
    n_points = len(X)
    k_max_actual = min(k_max, n_points)

    if k_max_actual < k_min:
        user_df["cluster"] = 0
        return imsi, user_df

    # 简单k-means（遍历K选最优SSE）
    best_k = k_min
    best_sse = np.inf
    for k in range(k_min, k_max_actual + 1):
        centroids = X[np.random.choice(n_points, k, replace=False)]
        for _ in range(50):
            dists = np.sqrt(((X[:, np.newaxis] - centroids) ** 2).sum(axis=2))
            labels = np.argmin(dists, axis=1)
            new_centroids = np.array([X[labels == i].mean(axis=0) if (labels ==
i).sum() > 0
                                      else centroids[i] for i in range(k)])
            if np.abs(new_centroids - centroids).max() < 1e-4:
                break
            centroids = new_centroids
        sse = sum(((X[labels == i] - centroids[i]) ** 2).sum() for i in range(k))
        if sse < best_sse:
            best_sse = sse
            best_k = k
```

```
            best_labels = labels

    user_df["cluster"] = best_labels
    return imsi, user_df


def demo_parallel_kmeans():
    """并行k-means聚类。"""
    print("\n" + "=" * 60)
    print("2. 并行k-means聚类")
    print("=" * 60)

    # 生成测试数据：50个用户，每个用户200个点
    n_users = 50
    n_points_per_user = 200
    data = []
    for i in range(n_users):
        center_lon = np.random.uniform(123.0, 124.0)
        center_lat = np.random.uniform(41.5, 42.5)
        for j in range(n_points_per_user):
            data.append({
                "imsi": f"user_{i}",
                "lon": center_lon + np.random.randn() * 0.01,
                "lat": center_lat + np.random.randn() * 0.01,
            })
    df = pd.DataFrame(data)
    print(f"  测试数据: {len(df)}行, {df['imsi'].nunique()}用户")

    # 串行k-means
    start = time.perf_counter()
    results_serial = []
    for imsi, user_df in df.groupby("imsi"):
        _, result = process_user_kmeans((imsi, user_df, 2, 10))
        results_serial.append(result)
    df_serial = pd.concat(results_serial, ignore_index=True)
    t_serial = time.perf_counter() - start

    # 并行k-means
```

```
    start = time.perf_counter()
    tasks = [(imsi, user_df, 2, 10) for imsi, user_df in df.groupby("imsi")]
    with Pool(processes=4) as pool:
        results_parallel = pool.map(process_user_kmeans, tasks)
    df_parallel = pd.concat([r[1] for r in results_parallel], ignore_index=True)
    t_parallel = time.perf_counter() - start

    print(f"  串行k-means: {t_serial:.2f}秒")
    print(f"  并行k-means(4进程): {t_parallel:.2f}秒")
    print(f"  加速比: {t_serial/t_parallel:.1f}倍")
    print(f"  结果行数一致: {len(df_serial) == len(df_parallel)}")


def demo_parallel_apply():
    """并行 apply。"""
    print("\n" + "=" * 60)
    print("3. 并行 apply（按行分组）")
    print("=" * 60)

    n = 100000
    df = pd.DataFrame({
        "a": np.random.rand(n),
        "b": np.random.rand(n),
        "group": np.random.randint(0, 100, n),
    })

    def process_group(group_df):
        """处理一个分组。"""
        return group_df["a"].sum() * group_df["b"].mean()

    # 串行
    start = time.perf_counter()
    results_serial = {}
    for group, group_df in df.groupby("group"):
        results_serial[group] = process_group(group_df)
    t_serial = time.perf_counter() - start

    # 并行
```

```
    start = time.perf_counter()
    tasks = [(group, group_df) for group, group_df in df.groupby("group")]
    with Pool(processes=4) as pool:
        results_parallel = pool.map(lambda x: (x[0], process_group(x[1])), tasks)
    results_parallel = dict(results_parallel)
    t_parallel = time.perf_counter() - start

    print(f"  串行: {t_serial:.4f}秒")
    print(f"  并行(4进程): {t_parallel:.4f}秒")
    print(f"  结果一致: {all(abs(results_serial[k] - results_parallel[k]) < 1e-6 for k
in results_serial)}")
    print(f"  注意：分组数太少时，并行开销可能大于收益")


if __name__ == "__main__":
    print("=" * 70)
    print("  并行计算实战")
    print("=" * 70)

    demo_basic_multiprocessing()
    demo_parallel_kmeans()
    demo_parallel_apply()

    print("\n" + "=" * 70)
    print("  并行计算实战完成！")
    print("=" * 70)
```

6.4 运行结果与解读

运行后展示3种并行计算：
1. 基础 multiprocessing：CPU密集型任务并行加速；
2. 并行k-means：按用户并行处理聚类；
3. 并行 apply：按分组并行处理。

> 关键理解：
> 1. multiprocessing 绕过 GIL，真正利用多核CPU；
> 2. 适合CPU密集型、独立任务、数据可分的场景；

> 3. Windows 下必须用 `if __name__ == "__main__":`；
> 4. 进程间通信有开销，任务太小不划算。

6.5 课时小结

- multiprocessing：多进程并行，绕过GIL；
- 适合CPU密集型、独立任务；
- Pool.map 最简单的并行方式；
- 注意Windows下必须用 `if __name__ == "__main__":`；
- 进程间通信有开销，任务太小不划算。

课时7：缓存优化

7.1 课时目标

掌握缓存优化技巧，避免重复计算。

7.2 知识点深度讲解

7.2.1 缓存优化的意义

缓存优化的意义：
1. **避免重复计算**：相同输入的计算结果缓存，下次直接使用；
2. **减少IO**：频繁读取的数据缓存到内存；
3. **提高响应速度**：缓存命中时几乎瞬间返回；
4. **降低CPU负载**：减少重复计算。

7.2.2 常见缓存方式

1. **字典缓存**：用字典存储计算结果；
2. **functools.lru_cache**：装饰器自动缓存；
3. **joblib.Memory**：磁盘缓存，适合大对象；
4. **pandas 缓存**：中间结果保存为CSV/parquet。

7.3 完整代码（缓存优化实战）

```
# ============================================================
# 课时7：缓存优化实战
# ============================================================

import time
import numpy as np
import pandas as pd
from functools import lru_cache


def demo_dict_cache():
    """字典缓存。"""
    print("=" * 60)
    print("1. 字典缓存")
    print("=" * 60)

    # 模拟耗时计算
    def expensive_calc(x):
        time.sleep(0.01)  # 模拟耗时
        return x ** 2 + np.sqrt(x)

    # 不缓存
    start = time.perf_counter()
    results1 = [expensive_calc(i % 100) for i in range(1000)]
    t1 = time.perf_counter() - start

    # 字典缓存
    cache = {}
    def cached_calc(x):
        if x not in cache:
            cache[x] = expensive_calc(x)
        return cache[x]

    start = time.perf_counter()
    results2 = [cached_calc(i % 100) for i in range(1000)]
    t2 = time.perf_counter() - start
```

```
    print(f"  不缓存: {t1:.2f}秒")
    print(f"  字典缓存: {t2:.2f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  缓存命中率: {(1000 - len(cache)) / 1000 * 100:.1f}%")
    print(f"  结果一致: {np.allclose(results1, results2)}")


def demo_lru_cache():
    """lru_cache 装饰器。"""
    print("\n" + "=" * 60)
    print("2. lru_cache 装饰器")
    print("=" * 60)

    @lru_cache(maxsize=128)
    def fibonacci(n):
        """计算斐波那契数列（递归）。"""
        if n < 2:
            return n
        return fibonacci(n - 1) + fibonacci(n - 2)

    # 不缓存的递归
    def fibonacci_no_cache(n):
        if n < 2:
            return n
        return fibonacci_no_cache(n - 1) + fibonacci_no_cache(n - 2)

    n = 35

    start = time.perf_counter()
    result1 = fibonacci_no_cache(n)
    t1 = time.perf_counter() - start

    start = time.perf_counter()
    result2 = fibonacci(n)
    t2 = time.perf_counter() - start

    print(f"  fibonacci({n}) = {result2}")
```

```
    print(f"  不缓存递归: {t1:.4f}秒")
    print(f"  lru_cache: {t2:.6f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  缓存信息: {fibonacci.cache_info()}")


def demo_distance_cache():
    """距离计算缓存（实际应用）。"""
    print("\n" + "=" * 60)
    print("3. 距离计算缓存（实际应用）")
    print("=" * 60)

    # 生成商业区数据（100个）
    n_biz = 100
    biz_lons = np.random.uniform(123.0, 124.0, n_biz)
    biz_lats = np.random.uniform(41.5, 42.5, n_biz)

    # 生成用户点（10000个）
    n_points = 10000
    point_lons = np.random.uniform(123.0, 124.0, n_points)
    point_lats = np.random.uniform(41.5, 42.5, n_points)

    def haversine(lon1, lat1, lon2, lat2):
        """Haversine距离。"""
        R = 6371.0
        lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
        dlat = lat2 - lat1
        dlon = lon2 - lon1
        a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
        return R * 2 * np.arcsin(np.sqrt(a))

    # 不缓存：每个点计算到所有商业区的距离
    start = time.perf_counter()
    nearest_biz1 = []
    for i in range(n_points):
        dists = [haversine(point_lons[i], point_lats[i], biz_lons[j], biz_lats[j])
                 for j in range(n_biz)]
        nearest_biz1.append(np.argmin(dists))
```

```
    t1 = time.perf_counter() - start

    # 向量化（不需要缓存，直接批量计算）
    start = time.perf_counter()
    # 广播：(10000,1) - (100,) -> (10000,100)
    dlon = np.radians(point_lons[:, np.newaxis] - biz_lons[np.newaxis, :])
    dlat = np.radians(point_lats[:, np.newaxis] - biz_lats[np.newaxis, :])
    a = np.sin(dlat/2)**2 + np.cos(np.radians(point_lats[:, np.newaxis])) * \
        np.cos(np.radians(biz_lons[np.newaxis, :])) * np.sin(dlon/2)**2
    dists = 6371.0 * 2 * np.arcsin(np.sqrt(a))
    nearest_biz2 = np.argmin(dists, axis=1)
    t2 = time.perf_counter() - start

    print(f"  逐行计算: {t1:.2f}秒")
    print(f"  向量化批量: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  结果一致: {(nearest_biz1 == nearest_biz2).all()}")


def demo_intermediate_result_cache():
    """中间结果缓存（保存为文件）。"""
    print("\n" + "=" * 60)
    print("4. 中间结果缓存（保存为文件）")
    print("=" * 60)

    import os
    cache_file = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result\cached_result.csv"

    def expensive_computation():
        """模拟耗时计算。"""
        time.sleep(2)  # 模拟耗时
        return pd.DataFrame({"a": np.random.rand(1000), "b": np.random.rand(1000)})

    # 第一次：计算并保存
    if os.path.exists(cache_file):
        os.remove(cache_file)
```

```
    start = time.perf_counter()
    if os.path.exists(cache_file):
        df = pd.read_csv(cache_file)
        print(f"  从缓存读取")
    else:
        df = expensive_computation()
        df.to_csv(cache_file, index=False)
        print(f"  计算并保存缓存")
    t1 = time.perf_counter() - start

    # 第二次：从缓存读取
    start = time.perf_counter()
    if os.path.exists(cache_file):
        df = pd.read_csv(cache_file)
        print(f"  从缓存读取")
    else:
        df = expensive_computation()
    t2 = time.perf_counter() - start

    print(f"  第一次（计算）: {t1:.2f}秒")
    print(f"  第二次（缓存）: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")


if __name__ == "__main__":
    print("=" * 70)
    print("  缓存优化实战")
    print("=" * 70)

    demo_dict_cache()
    demo_lru_cache()
    demo_distance_cache()
    demo_intermediate_result_cache()

    print("\n" + "=" * 70)
    print("  缓存优化实战完成！")
    print("=" * 70)
```

7.4 运行结果与解读

运行后展示4种缓存优化：
1. 字典缓存：避免重复计算；
2. lru_cache：装饰器自动缓存，斐波那契数列加速巨大；
3. 距离计算缓存：向量化批量计算替代逐行；
4. 中间结果缓存：保存为文件，避免重复计算。

7.5 课时小结

- 缓存优化：字典缓存、lru_cache、中间结果文件缓存；
- lru_cache 装饰器最简单，适合函数级缓存；
- 中间结果保存为CSV/parquet，适合大对象；
- 向量化批量计算是最好的"缓存"——一次计算所有结果。

课时8：系统性能优化实战

8.1 课时目标

综合运用各种优化技巧，优化整个系统。

8.2 知识点深度讲解

8.2.1 系统优化策略

系统优化的策略：
1. **先测量后优化**：用 cProfile 找出瓶颈；
2. **优先优化瓶颈**：80%时间花在20%代码上；
3. **向量化优先**：能用向量化就不用循环；
4. **缓存重复计算**：相同输入的结果缓存；
5. **并行处理**：独立任务用多进程；
6. **内存优化**：选择合适数据类型。

8.2.2 本系统的优化点

1. **数据清洗**：向量化字符串操作、merge替代循环；
2. **阶段1**：向量化距离计算、groupby替代循环；
3. **阶段2**：并行k-means（按用户）、向量化距离矩阵；
4. **阶段3**：向量化距离计算、减少循环；

5. **出行目的/方式**：向量化空间匹配、merge替代循环；
6. **实时热力**：向量化滑动窗口、减少循环。

8.3 完整代码（系统优化前后对比）

```
# ============================================================
# 课时8：系统性能优化实战
# ============================================================

import time
import numpy as np
import pandas as pd


def optimize_data_clean():
    """优化数据清洗。"""
    print("=" * 60)
    print("1. 数据清洗优化")
    print("=" * 60)

    # 生成测试数据
    n = 100000
    df = pd.DataFrame({
        "imsi": [f"4600000{np.random.randint(100000, 999999)}" for _ in range(n)],
        "time": [f"20181003{np.random.randint(0, 23):02d}{np.random.randint(0,
59):02d}00" for _ in range(n)],
        "lac_id": [str(np.random.randint(1000, 9999)) for _ in range(n)],
        "cell_id": [str(np.random.randint(10000, 99999)) for _ in range(n)],
    })

    # 原始方式：逐行判断
    start = time.perf_counter()
    df1 = df.copy()
    mask = []
    for _, row in df1.iterrows():
        mask.append(not (row["lac_id"] == "" or row["cell_id"] == ""))
    df1 = df1[mask]
    t1 = time.perf_counter() - start

    # 优化方式：向量化
    start = time.perf_counter()
```

```
    df2 = df.copy()
    df2 = df2[(df2["lac_id"] != "") & (df2["cell_id"] != "")]
    t2 = time.perf_counter() - start

    print(f"  逐行判断: {t1:.4f}秒")
    print(f"  向量化: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")


def optimize_stage1():
    """优化阶段1（速度计算）。"""
    print("\n" + "=" * 60)
    print("2. 阶段1优化（速度计算）")
    print("=" * 60)

    n = 10000
    df = pd.DataFrame({
        "imsi": np.random.choice([f"user_{i}" for i in range(100)], n),
        "time": pd.date_range("2018-10-03", periods=n,
freq="min").strftime("%Y%m%d%H%M%S"),
        "lon": np.random.uniform(123.0, 124.0, n),
        "lat": np.random.uniform(41.5, 42.5, n),
    })
    df = df.sort_values(["imsi", "time"]).reset_index(drop=True)

    # 原始方式：逐行计算
    start = time.perf_counter()
    df1 = df.copy()
    speeds = []
    prev_imsi = None
    prev_time = None
    prev_lon = None
    prev_lat = None
    for _, row in df1.iterrows():
        if row["imsi"] == prev_imsi and prev_time is not None:
            from math import radians, sin, cos, sqrt, atan2
            R = 6371.0
            dlat = radians(row["lat"] - prev_lat)
```

```
            dlon = radians(row["lon"] - prev_lon)
            a = sin(dlat/2)**2 + cos(radians(prev_lat)) * cos(radians(row["lat"])) *
sin(dlon/2)**2
            dist = R * 2 * atan2(sqrt(a), sqrt(1-a))
            time_diff = (pd.to_datetime(row["time"]) -
pd.to_datetime(prev_time)).total_seconds()
            speed = dist * 3600 / time_diff if time_diff > 0 else 0
            speeds.append(speed)
        else:
            speeds.append(np.nan)
        prev_imsi = row["imsi"]
        prev_time = row["time"]
        prev_lon = row["lon"]
        prev_lat = row["lat"]
    df1["speed"] = speeds
    t1 = time.perf_counter() - start

    # 优化方式：向量化
    start = time.perf_counter()
    df2 = df.copy()
    df2["time_dt"] = pd.to_datetime(df2["time"], format="%Y%m%d%H%M%S")
    df2["time_diff"] = df2.groupby("imsi")["time_dt"].diff().dt.total_seconds()
    df2["prev_lon"] = df2.groupby("imsi")["lon"].shift(1)
    df2["prev_lat"] = df2.groupby("imsi")["lat"].shift(1)
    lons = np.radians(df2["lon"].values)
    lats = np.radians(df2["lat"].values)
    prev_lons = np.radians(df2["prev_lon"].values)
    prev_lats = np.radians(df2["prev_lat"].values)
    dlat = lats - prev_lats
    dlon = lons - prev_lons
    a = np.sin(dlat/2)**2 + np.cos(prev_lats) * np.cos(lats) * np.sin(dlon/2)**2
    df2["dist"] = 6371.0 * 2 * np.arcsin(np.sqrt(a))
    df2["speed"] = df2["dist"] * 3600 / df2["time_diff"]
    t2 = time.perf_counter() - start

    print(f"  逐行计算: {t1:.2f}秒")
    print(f"  向量化: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
```

```
def optimize_spatial_match():
    """优化空间匹配。"""
    print("\n" + "=" * 60)
    print("3. 空间匹配优化")
    print("=" * 60)

    n_points = 10000
    n_pois = 100
    points = pd.DataFrame({
        "lon": np.random.uniform(123.0, 124.0, n_points),
        "lat": np.random.uniform(41.5, 42.5, n_points),
    })
    pois = pd.DataFrame({
        "name": [f"POI_{i}" for i in range(n_pois)],
        "lon": np.random.uniform(123.0, 124.0, n_pois),
        "lat": np.random.uniform(41.5, 42.5, n_pois),
    })

    # 原始方式：双重循环
    start = time.perf_counter()
    nearest1 = []
    for _, point in points.iterrows():
        min_dist = np.inf
        min_idx = -1
        for j, poi in pois.iterrows():
            d = np.sqrt((point["lon"] - poi["lon"])**2 + (point["lat"] -
poi["lat"])**2)
            if d < min_dist:
                min_dist = d
                min_idx = j
        nearest1.append(min_idx)
    t1 = time.perf_counter() - start

    # 优化方式：广播向量化
    start = time.perf_counter()
    point_lons = points["lon"].values[:, np.newaxis]
```

```
    point_lats = points["lat"].values[:, np.newaxis]
    poi_lons = pois["lon"].values[np.newaxis, :]
    poi_lats = pois["lat"].values[np.newaxis, :]
    dists = np.sqrt((point_lons - poi_lons)**2 + (point_lats - poi_lats)**2)
    nearest2 = np.argmin(dists, axis=1)
    t2 = time.perf_counter() - start

    print(f"  双重循环: {t1:.2f}秒")
    print(f"  广播向量化: {t2:.4f}秒")
    print(f"  加速比: {t1/t2:.1f}倍")
    print(f"  结果一致: {(nearest1 == nearest2).all()}")


def optimization_summary():
    """优化总结。"""
    print("\n" + "=" * 60)
    print("4. 系统优化总结")
    print("=" * 60)

    print("""
  优化技巧                适用场景                    加速比
  ──────────────────────────────────────────────────────────
  向量化运算              数组/列运算                 10-100x
  merge替代循环           数据关联匹配                5-50x
  groupby替代循环         分组统计                    5-20x
  category类型            低基数字符串列              内存节省90%+
  数值类型优化            整数/浮点数                 内存节省50-75%
  lru_cache               重复函数调用                10-1000x
  multiprocessing         CPU密集型独立任务           2-8x(取决于核心数)
  中间结果文件缓存        耗时计算结果                10-100x
  分块读取                大文件处理                  避免OOM
  ──────────────────────────────────────────────────────────

  优化原则:
  1. 先测量后优化，不要凭感觉
  2. 优先优化瓶颈（80%时间在20%代码）
  3. 向量化优先，能用向量化就不用循环
  4. 保持正确性，优化不能改变结果
```

```
  5. 对比测试，优化前后都要验证
    """)


if __name__ == "__main__":
    print("=" * 70)
    print("  系统性能优化实战")
    print("=" * 70)

    optimize_data_clean()
    optimize_stage1()
    optimize_spatial_match()
    optimization_summary()

    print("\n" + "=" * 70)
    print("  系统性能优化实战完成！")
    print("=" * 70)
```

8.4 运行结果与解读

运行后展示3个系统优化案例和优化总结：
1. 数据清洗优化：向量化替代逐行判断；
2. 阶段1优化：向量化速度计算；
3. 空间匹配优化：广播向量化替代双重循环；
4. 优化总结：各种优化技巧的适用场景和加速比。

8.5 课时小结

- 系统优化：先测量后优化、优先优化瓶颈、向量化优先；
- 数据清洗：向量化字符串操作、merge替代循环；
- 阶段1：向量化距离计算、groupby替代循环；
- 空间匹配：广播向量化替代双重循环；
- 优化技巧总结：向量化(10-100x)、merge(5-50x)、category(内存90%+)、lru_cache(10-1000x)、multiprocessing(2-8x)。

三、本日小结

1. **性能测试基础**：time、timeit、cProfile、memory_profiler；

2. **系统瓶颈分析**：全流程计时→函数级分析→内存分析；
3. **向量化优化**：numpy/pandas 向量化替代 for 循环，加速几十倍；
4. **pandas 优化**：apply→向量化、iterrows→itertuples→向量化、merge替代循环；
5. **内存优化**：数值类型、category、downcast、分块读取；
6. **并行计算**：multiprocessing 多进程，绕过GIL；
7. **缓存优化**：字典缓存、lru_cache、中间结果文件缓存；
8. **系统优化实战**：综合运用各种优化技巧，优化数据清洗、阶段1、空间匹配。

四、本日作业

1. 用 cProfile 分析 main.py 的性能瓶颈，找出最慢的3个函数；
2. 把 tr_stage2.py 中的 k-means 改为并行版本（按用户并行）；
3. 把 purpose.py 中的空间匹配改为广播向量化版本；
4. 对比优化前后的执行时间，记录加速比；
5. 预习 Day13：思考"如何对系统进行测试和调试？"